P.Vishwa Nandan(192473015)

TASK 6 – Design the Inheritance Hierarchy

Marks: 4

This task is about using **Inheritance in Java** to represent different vehicle types.

The assignment asks us to design a superclass and subclasses, explain super, show constructor execution order, draw a UML class diagram and compare inheritance with composition. CO1 JAVA

1. Superclass and Subclasses

Explanation

Inheritance allows one class to acquire the properties and methods of another class.

In our parking system, all vehicles have common information:

- Registration number.
- Vehicle details.
- Parking fee calculation.

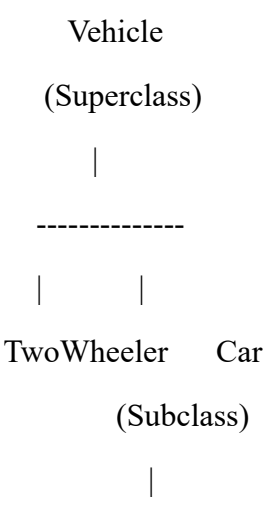
Instead of writing these properties separately for every vehicle, we create one parent class called Vehicle.

Superclass: Vehicle

Subclasses:

1. TwoWheeler
2. Car
3. ElectricCar

The inheritance structure is:



ElectricCar

Here:

- TwoWheeler inherits from Vehicle.
- Car inherits from Vehicle.
- ElectricCar inherits from Car.

Types of Inheritance Used

1. Hierarchical Inheritance

Multiple subclasses inherit from one superclass.

Example: TwoWheeler and Car inherit from Vehicle.

2. Multilevel Inheritance

A subclass inherits from another subclass.

Example: ElectricCar inherits from Car, which inherits from Vehicle.

2. What Is Inherited and What Is Added?

The following properties and behaviours are inherited or added in each class.

Vehicle (Superclass):

- Registration number.
- Vehicle details.
- Common vehicle methods.

TwoWheeler (Subclass):

- Inherits registration number from Vehicle.
- Adds two-wheeler-specific parking rate.

Car (Subclass):

- Inherits registration number from Vehicle.
- Adds car-specific parking rate.

ElectricCar (Subclass):

- Inherits properties from Car and Vehicle.
- Adds electric vehicle charging functionality.

This structure reduces code repetition and improves reusability.

3. Use of super Keyword

The super keyword refers to the immediate parent class.

It is mainly used in three ways.

A. Accessing Parent Class Fields

```
class Vehicle {
   String regNo = "TN01AB1234";
}

class Car extends Vehicle {

   void display() {
      System.out.println(super.regNo);
   }
}
```

Here, super.regNo accesses the parent class variable.

B. Calling Parent Class Methods

```
class Vehicle {

   void display() {
      System.out.println("Vehicle details");
   }
}

class Car extends Vehicle {

   void show() {
      super.display();
      System.out.println("Car details");
   }
}
```

Here, super.display() calls the parent class method.

C. Calling Parent Class Constructor

```
class Vehicle {

    Vehicle(String regNo) {
        System.out.println("Vehicle: " + regNo);
    }
}

class Car extends Vehicle {

    Car(String regNo) {
        super(regNo);
        System.out.println("Car created");
    }
}
```

Here, super(regNo) calls the parent class constructor.

4. Constructor Execution Order for ElectricCar

When an ElectricCar object is created, constructors execute from the parent class to the child class.

Consider:

```
class Vehicle {

    Vehicle() {
        System.out.println("Vehicle constructor");
    }
}

class Car extends Vehicle {
```

```
    Car() {
        super();
        System.out.println("Car constructor");
    }
}

class ElectricCar extends Car {

    ElectricCar() {
        super();
        System.out.println("ElectricCar constructor");
    }
}

class Main {

    public static void main(String[] args) {

        ElectricCar e = new ElectricCar();
    }
}
```

Output:

Vehicle constructor

Car constructor

ElectricCar constructor

Explanation:

1. First, the Vehicle constructor executes.
2. Next, the Car constructor executes.

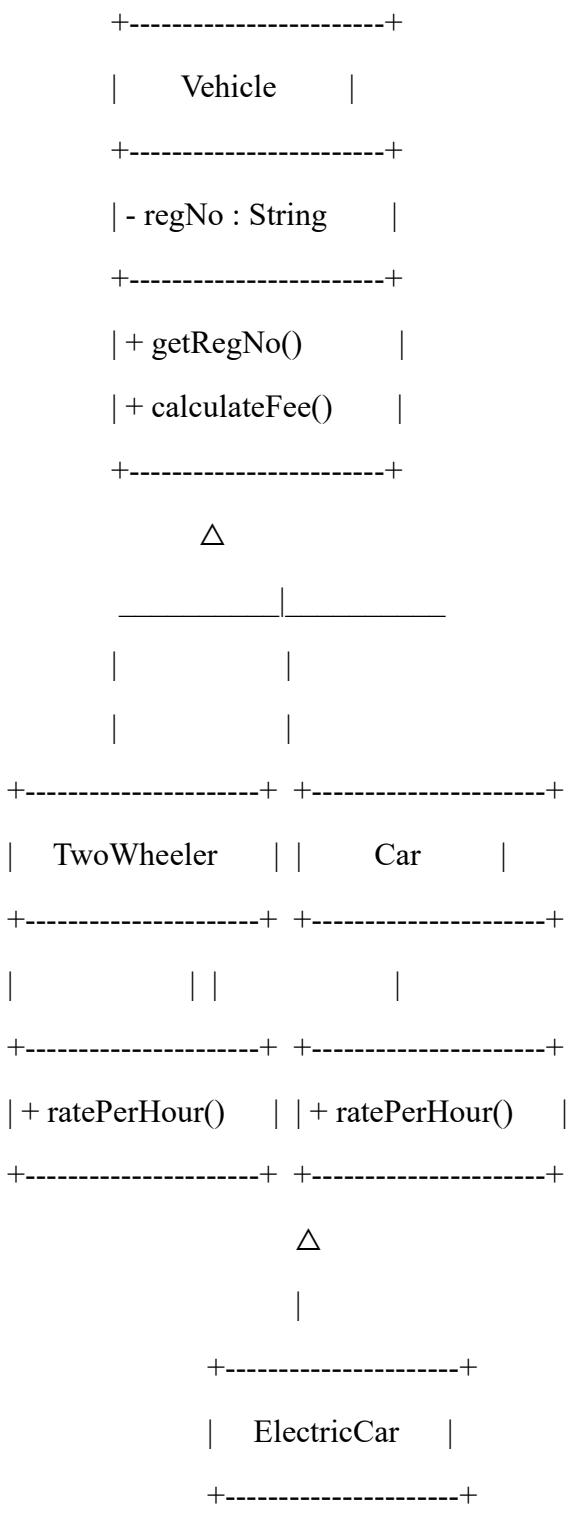
3. Finally, the ElectricCar constructor executes.

This happens because the child constructor calls its parent constructor using super().

The implicit Object superclass constructor executes before the Vehicle constructor.

5. UML Class Diagram

The following UML diagram represents the vehicle inheritance hierarchy.



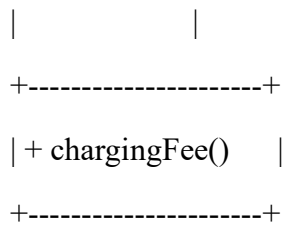


Diagram explanation:

- Vehicle is the parent class.
- TwoWheeler and Car inherit from Vehicle.
- ElectricCar inherits from Car.
- Common properties are maintained in the Vehicle class.

6. Alternative Concept and Complexity Analysis

Instead of inheritance, we can use **Composition**.

Composition means creating a class that contains an object of another class.

For example, instead of creating separate subclasses for Car and TwoWheeler, we can create one Vehicle class that contains a VehicleType object.

A. Composition Approach

Example:

```
class VehicleType {

    double rate;

    VehicleType(double rate) {
        this.rate = rate;
    }
}

class Vehicle {

    String regNo;
    VehicleType type;
```

```
    Vehicle(String regNo, VehicleType type) {
        this.regNo = regNo;
        this.type = type;
    }

    double calculateFee(int hours) {
        return hours * type.rate;
    }
}

class Main {

    public static void main(String[] args) {

        VehicleType carType = new VehicleType(30);

        Vehicle v = new Vehicle("TN01AB1234", carType);

        System.out.println("Parking Fee: Rs. " + v.calculateFee(2));
    }
}
```

Output:

Parking Fee: Rs. 60.0

Here, Vehicle contains a VehicleType object that stores the hourly rate.

B. Comparison Between Inheritance and Composition

Inheritance:

- Creates separate subclasses for different vehicle types.
- Uses method overriding to define different parking rates.

- Adding a new vehicle type requires creating a new subclass.
- Each vehicle object has constant-sized object storage overhead.

Composition:

- Uses one Vehicle class and a separate VehicleType object.
- Vehicle contains the pricing information through composition.
- Adding a new vehicle type requires creating a new VehicleType configuration.
- Multiple vehicles can share the same VehicleType object.

C. Complexity Analysis

Inheritance Approach:

- Fee calculation time: O(1).
- Space per vehicle object: O(1).

Composition Approach:

- Fee calculation time: O(1), assuming direct access to the VehicleType object.
- Space per vehicle object: O(1), including its reference to VehicleType.
- Additional VehicleType objects require O(k) space for k vehicle types.

Here, k represents the number of vehicle types.

Both approaches provide constant-time fee calculation. Composition makes adding new vehicle configurations easier, while inheritance provides clear class relationships and supports specialized vehicle behaviour.

7. Conclusion

Inheritance is used to organize different vehicle types under a common Vehicle superclass. Hierarchical and multilevel inheritance improve code reusability. The super keyword allows subclasses to access parent constructors, fields and methods. Composition is an alternative approach that makes adding new vehicle types and configurations easier.